VOL. 19

TypeScript

Static types for JavaScript, from annotations to generics and compiler options

Source	typescriptlang.org/docs
Generated	2026-10-10
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 19 - TypeScript

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 5

Chapter map 5

Chapter 1 - What TypeScript is and how the toolchain runs 6

1.1 Summary 6

1.2 Concepts 7

What TypeScript actually adds 7

Type erasure 7

The toolchain 8

Four ways to call the compiler 8

A first `tsconfig.json` 8

Strictness in one paragraph 9

1.3 Examples 9

1.4 Common mistakes and best practices 11

1.5 Practice exercises 11

Solutions 11

1.6 Mini test 12

Answer key and explanations 12

Chapter 2 - Types, annotations and inference 12

2.1 Summary 12

2.2 Concepts 13

The names for JavaScript's values 13

Where annotations may go 13

Arrays, tuples and readonly 14

`any` versus `unknown` 14

Inference: what the compiler works out for you 14

Literal types and widening 14

`null`, `undefined` and strictness 15

2.3 Examples 15

2.4 Common mistakes and best practices 17

2.5 Practice exercises 17

Solutions 18

2.6 Mini test 18

Answer key and explanations 18

Chapter 3 - Unions, literals and narrowing . . . 19
3.1 Summary . . . 19
3.2 Concepts . . . 19
Writing unions . . . 19
The guards . . . 20
Discriminated unions . . . 20
Exhaustiveness and `never` . . . 20
Where narrowing stops . . . 21
Assertions . . . 21
3.3 Examples . . . 21
3.4 Common mistakes and best practices . . . 24
3.5 Practice exercises . . . 25
Solutions . . . 25
3.6 Mini test . . . 26
Answer key and explanations . . . 26
Chapter 4 - Functions and overloads . . . 26
4.1 Summary . . . 26
4.2 Concepts . . . 27
One function, two descriptions . . . 27
Parameters . . . 27
Return types . . . 27
Overloads . . . 27
Callbacks and contextual typing . . . 28
4.3 Examples . . . 28
4.4 Common mistakes and best practices . . . 30
4.5 Practice exercises . . . 31
Solutions . . . 31
4.6 Mini test . . . 32
Answer key and explanations . . . 32
Chapter 5 - Interfaces, type aliases and object shapes . . . 32
5.1 Summary . . . 32
5.2 Concepts . . . 33
Writing a shape . . . 33
Interface or type alias? . . . 33
Extending and intersecting . . . 34

Index signatures . . . 34
Excess property checking . . . 34
5.3 Examples . . . 34
5.4 Common mistakes and best practices . . . 37
5.5 Practice exercises . . . 37
Solutions . . . 38
5.6 Mini test . . . 38
Answer key and explanations . . . 39

How to use this volume

Prerequisites. Working modern JavaScript (Vol. 03): `let` and `const`, arrow functions, destructuring, template literals, classes, ES modules, promises and `async`/`await`. No previous TypeScript is assumed, and no other language is required.

Estimated study time. About 25 hours: two to three hours per chapter plus the time you spend changing the examples yourself. Chapters 1 to 6 are the core you will use every day; chapters 7 to 11 cover classes, enums and type-level programming; chapters 12 and 13 deal with real projects.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions, and a mini test with an answer key. A cheat sheet and a 25 question final test close the volume.

A note on running the code. The machine used to prepare this volume has Node v22 but no TypeScript compiler installed, so every example that prints output was executed with Node's built-in type stripping, which removes the type annotations and runs the remaining JavaScript. Where a construct is not erasable syntax - enums, namespaces, parameter properties - the example says so and is either run with `node --experimental-transform-types` or explained in prose. Anything that only a type checker can observe (an error message, a type alias resolving to a shape) is described rather than printed, because no compiler was available locally. Commands that belong to `tsc` are shown as commands, and their result is stated in words instead of being faked.

shell

```shell
# any example in this volume, when the annotations can simply be removed
node example.ts

# enums, namespaces and parameter properties need the transform mode
node --experimental-transform-types example.ts

# a full check with the real compiler, when you have it installed
npx tsc --noEmit
```

output

```
hello TypeScript
```

Version numbers change with every release; the shape of the output does not.

About the source. The syllabus follows the official documentation hub at typescriptlang.org, using the reading order of the "Get Started" pages and the second-edition Handbook, plus the Reference pages for enums, decorators, utility types and project configuration. Topic names are paraphrased; no sentences, illustrations or full code listings were copied.

Chapter map

#	Chapter	You will learn
1	What TypeScript is and how the toolchain runs	types, type erasure, `tsc`, `tsconfig.json`, strictness
2	Types, annotations and inference	primitives, arrays, tuples, `any` versus `unknown`, inference, widening
3	Unions, literals and narrowing	union types, type guards, discriminated unions, `never` and exhaustiveness
4	Functions and overloads	function types, optional and rest parameters, callbacks, overload signatures
5	Interfaces, type aliases and object shapes	shapes, optional and readonly members, index signatures, merging
6	Generics	type parameters, constraints, defaults, generic classes and functions
7	Classes, mixins and a decorator overview	modifiers, parameter properties, abstract classes, mixin functions, decorators
8	Enums, const assertions, symbols and generators	numeric and string enums, `as const`, iterators, generator functions
9	Utility types and deriving types from values	`Partial`, `Pick`, `Omit`, `Record`, `keyof`, `typeof`, indexed access, mapped types
10	Conditional types, template literal types and `infer`	type-level if and else, distribution over unions, string pattern types
11	Modules and project structure	ES modules, type-only imports, resolution modes, project references
12	tsconfig, tooling and migrating a JavaScript project	strict flags, `allowJs` and `checkJs`, JSDoc, incremental migration
13	TypeScript in the wild: React and Node	typed components and props, typed request handlers, ambient declarations
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - What TypeScript is and how the toolchain runs

1.1 Summary

- TypeScript is JavaScript with a type system bolted on, not a different language you have to learn from zero.
- A type is a description of a value used while the program is being written; the compiler checks it and then throws it away.

- The compiled output is ordinary JavaScript, so every TypeScript program is still a JavaScript program that runs anywhere JavaScript runs.
- The compiler is a command line program called `tsc`; in practice you drive it with a `tsconfig.json` file that lists which files to check and which options are enabled.
- `tsc --noEmit` checks a project without writing any output, which is what continuous integration usually runs.
- `tsc -w` and the editor's language service watch your files and report problems as you type, using the same engine as the command line tool.
- Strictness is opt-in through flags, and the `strict` family turns the important ones on at once - new projects should start with it.
- TypeScript adds types and nothing else: it does not change `this`, does not add runtime checks and does not change how the code executes.
- Because types are erased, a library running in production never sees an annotation, which is why type errors never break a deployed program.
- The exact behaviour of a flag or a piece of syntax depends on the compiler version, so it pays to know which version a project uses before copying configuration from the internet.

1.2 Concepts

What TypeScript actually adds

JavaScript already has types at runtime - numbers, strings, objects, functions. What it lacks is a way to write those types down and have them checked before the code runs. TypeScript adds exactly that: a notation for describing the shape of your values, and a checker that complains when the code and the notation disagree.

Nothing else changes. The syntax is JavaScript plus annotations, the standard library is the JavaScript one plus type declarations for it, and the output is JavaScript.

output

```
source .ts file          tsc / editor service          running program
+----------------+        +--------------------+        +---------------+
| types + values | -----> | remove types,      | -----> | plain JS, no  |
| user.ts        |        | report mismatches  |        | type checks   |
+----------------+        +--------------------+        +---------------+
```

Type erasure

Removing the types is called erasure, and it is the single most important idea in this volume. Consider this function:

TypeScript

```ts
function area(width: number, height: number): number {
  return width * height;
}
```

The annotation `: number` on the parameters, the `: number` after the parameter list and the type alias declarations all disappear. What is left is:

JavaScript

```
function area(width, height) {
  return width * height;
}
```

Two consequences follow. First, nothing at runtime can inspect a type, so a program cannot ask whether an argument "was declared as a number". Second, a type error is a build-time event: the program either never starts, or it starts and is guaranteed to be the untyped JavaScript you already know.

The toolchain

Tool	What it does
`tsc`	the reference compiler: checks types and writes JavaScript
TS Language Service	the engine editors use for completion and inline errors
`tsconfig.json`	the project description: files, options, output layout
A package manager	installs TypeScript itself and the `@types/*` packages
A bundler or runtime	runs the emitted JavaScript, often with its own TS support

TypeScript is normally installed per project, not globally, so that everyone on a team checks with the same version. `npx tsc --version` tells you what you have.

Four ways to call the compiler

Command	Effect
`tsc file.ts`	compiles one file and writes `file.js` next to it
`tsc` with a `tsconfig.json`	checks the whole project and writes to `outDir` if set
`tsc --noEmit`	checks only, writes nothing - the CI command
`tsc -w`	checks, then watches and re-checks on every save

`tsc --init` writes a starter `tsconfig.json` with most options commented out, which is a good way to see which options exist.

A first `tsconfig.json`

JSON

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "noEmit": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

Option	Meaning
`target`	which JavaScript version the output uses
`module`	the module syntax of the output
`moduleResolution`	how an import path is resolved to a file
`strict`	enables the family of strictness checks
`noEmit`	check only, do not write JavaScript
`skipLibCheck`	skip type checking of declaration files in `node_modules`
`include` / `exclude`	which files belong to the project

The resolution option is the one that trips people up most: `Bundler` assumes a bundler resolves imports, while `NodeNext` follows Node's rules for `.js` extensions in import specifiers. Pick the one that matches how your code is actually run.

Strictness in one paragraph

By default the compiler is deliberately permissive so that existing JavaScript keeps compiling. `strict` turns on the checks that catch real bugs: `noImplicitAny` refuses parameters whose type cannot be inferred, `strictNullChecks` stops `null` and `undefined` from hiding in every type, and `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, `noImplicitThis` and `useUnknownInCatchVariables` complete the set. Every one of them can also be switched on individually, which is how you migrate a project one check at a time.

1.3 Examples

Example 1: a first file, executed with Node's type stripping.

TypeScript

```ts
type Item = { name: string; price: number };

function total(items: Item[]): number {
  return items.reduce((sum, item) => sum + item.price, 0);
}

const cart = [
  { name: "notebook", price: 12 },
  { name: "pen", price: 3 },
];

console.log(total(cart));
```

output

```
15
```

The `type Item` line and the annotations are removed before execution; the program that runs is plain JavaScript.

Example 2: the same file after erasure, run as pure JavaScript.

JavaScript

```
function total(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}

const cart = [
  { name: "notebook", price: 12 },
  { name: "pen", price: 3 },
];

console.log(total(cart));
```

output

```
15
```

Same output, same file size, no checks. This is what `tsc` writes to disk, and it is why TypeScript never needs a runtime library.

Example 3: what the checker refuses. The line below does not run to completion in a check-only tool such as `tsc --noEmit`, because a string literal is being assigned to a variable declared as `number`.

TypeScript

```
const major: number = "five";
```

output

```
tsc reports an assignment error on the line above and stops; the wording and
the error code vary between compiler versions. Node's type stripping ignores
types entirely, so the same line runs there and prints "five". That gap is
exactly why you run a real check instead of trusting a type-stripping runner.
```

Example 4: the project file, again, with a comment about what changes when you want output.

JSON

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "strict": true,
    "noEmit": true,
    "skipLibCheck": true
  },
  "include": ["src"]
}
```

output

```
(no console output; this file is read by tsc and by the editor's language
service, never executed)
```

Example 5: the three commands you will type most often.

shell

```
npx tsc --noEmit   # check the whole project, write nothing
npx tsc -w         # check, then keep watching for changes
npx tsc --init     # create a starter tsconfig.json
```

output

```
npx: installed 1 in 1s        (only the first time, then cached)
tsc: no errors                 (--noEmit prints nothing when the project is clean)
```

The exact output of the install line depends on your package manager and version; the important part is that a clean check is silent.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assuming a type error will show up at runtime	Types are erased, so nothing at runtime ever looks at them	Treat `tsc --noEmit` as part of your test run
Installing TypeScript globally only	Different machines then check with different versions and disagree	Install it per project and use `npx tsc`
Trusting a type-stripping runner as a check	Node and bundlers remove types without checking them	Use the compiler or the editor service for checking
Copying a `tsconfig.json` from a blog post wholesale	Options such as `moduleResolution` must match how your code is actually run	Start from `tsc --init`, then change one option at a time
Leaving `strict` off in a new project	You inherit `any` bugs that are cheap to prevent and expensive to find later	Turn `strict` on from the first commit
Mixing `tsc` output and source in one folder	You end up editing generated files by mistake	Set `rootDir` and `outDir`, or use `noEmit` and let a bundler compile

1.5 Practice exercises

1. Write the `tsconfig.json` for a project whose sources live in `src`, that is checked with all strictness flags on and produces no output files.
2. A colleague says "TypeScript would slow our app down at runtime". Explain in two sentences what actually happens to the types.
3. Which command would you add to a continuous integration pipeline to make sure a type error blocks a merge?
4. Name the file a new project uses to tell the compiler which files to check, and one option in it that has nothing to do with checking.

Solutions

1. `{"compilerOptions": {"strict": true, "noEmit": true}, "include": ["src"]}` is enough; `skipLibCheck: true` is a useful extra once you depend on `@types` packages.

2. Types exist only while the program is being written and checked. They are removed before the code runs, so the deployed program is ordinary JavaScript and executes at exactly the same speed as before.
3. `tsc --noEmit` (or the project's own typecheck script, which usually calls it). It prints nothing on success and fails the build on a type error.
4. `tsconfig.json`; `outDir`, `declaration` or `sourceMap` describe what is written to disk rather than what is checked.

1.6 Mini test

1. What does the compiler do with type annotations before the program runs?
2. True or false: a TypeScript program needs a small runtime library to handle its types.
3. Which option makes the compiler check a project without writing any JavaScript files?
4. Which flag turns on the whole family of strictness checks?
5. What does `moduleResolution: "Bundler"` assume about your project?
6. Which command writes a starter `tsconfig.json`?
7. Why can a type error never be the cause of a crash in production?

Answer key and explanations

1. It removes them. Erasure happens after checking, and the emitted JavaScript contains no annotations at all.
2. False. Types are a compile-time concept, so the output is plain JavaScript and runs without any extra library.
3. `noEmit`, used as `tsc --noEmit`.
4. `strict`. Individual flags such as `noImplicitAny` can also be set on their own when migrating.
5. That a bundler resolves the import specifiers, so extensionless imports are fine and Node's rules do not apply.
6. `tsc --init`.
7. Because by the time the program is deployed there are no types left to crash on: a type error either stops the build or does not exist at runtime.

Chapter 2 - Types, annotations and inference

2.1 Summary

- TypeScript gives names to the values JavaScript already has: `string`, `number`, `boolean`, objects, arrays and functions.
- An annotation is a colon plus a type name after a variable, a parameter or a return position, and the compiler then holds you to it.

- Inference means the compiler works out the type from the value, so most local variables need no annotation at all.
- `any` switches checking off for a value and lets the problem spread; `unknown` accepts anything but forces you to narrow before using it.
- Arrays are written `T[]` or `Array<T>`, and a tuple fixes both the length and the type of each position.
- A union type lets one value hold more than one kind of thing, written with a vertical bar.
- With `strictNullChecks` enabled, `null` and `undefined` only appear in a type where you put them; without it they hide in every type.
- A literal type describes one exact value, and `let` widens a literal to its base type while `const` keeps the exact value.
- `readonly` marks members that may not be reassigned after creation, which documents intent without changing runtime behaviour.
- The habit worth building early: annotate the boundaries of your code and let inference handle the inside.

2.2 Concepts

The names for JavaScript's values

Annotation	Holds	Example
`string`	text	`"Ada"`
`number`	any numeric value, integer or float	`3.14`
`boolean`	`true` or `false`	`false`
`null` / `undefined`	the two empty values	`null`
`object`	any non-primitive value	`{ id: 1 }`
`array of T`	a list of values of type T	`[1, 2, 3]`
`function`	any callable value	`(x) => x`

`object` is broader than it looks and narrower than it sounds: it means "not a primitive", so `{ id: 1 }` and `[]` both fit. When you want to describe a shape, write the shape instead.

Where annotations may go

TypeScript

```typescript
let total: number = 0;
const label: string = "sum";
function twice(value: number): number {
  return value * 2;
}
const add = (a: number, b: number): number => a + b;
```

Every annotation is a promise: "I will only ever put a number here". If code somewhere else puts a string there, the compiler stops you at build time. Note that nothing in this list changes

what the program does - it is still the same arithmetic.

Arrays, tuples and readonly

An array type is a type plus a suffix or a generic call:

TypeScript

```ts
const names: string[] = ["Ada", "Grace"];
const scores: Array<number> = [90, 85];
const mixed: (string | number)[] = ["Ada", 1];
```

A tuple is a list whose length and per-position types are known:

TypeScript

```ts
const point: [number, number] = [4, 9];
const entry: [string, number] = ["id-7", 7];
```

Tuple positions can also be optional or rest-shaped, so `[number, ...string[]]` means "a number, then any number of strings". Marking an array or a member `readonly` forbids reassignment; it is a compile-time promise, not a frozen object.

`any` versus `unknown`

Question	any	unknown
Can any value be assigned to it?	yes	yes
Can you use the value directly?	yes, with no checks	no, you must narrow first
Does it affect neighbouring values?	yes, it spreads silently	no, it stays contained
Good for	a quick escape hatch	data whose shape you do not know, such as parsed JSON

`unknown` is the type of `JSON.parse`. You accept the value happily, but you have to ask what it is before you touch its members.

Inference: what the compiler works out for you

A variable declared with a value inherits that value's type:

TypeScript

```ts
const count = 3;          // number
const ratio = 0.5;        // number
const names = ["a", "b"]; // string[]
const flags = [1, "a"];   // (string | number)[]
```

Function return types are inferred too, along with the types of callback parameters from the function you pass them to. Inference is not magic: it is the compiler reading your code. When you find yourself guessing, write the annotation - it costs one word and removes a question.

Literal types and widening

A literal type names a single value. Assignment from a literal widens for `let`, because the variable is allowed to change:

Declaration	Inferred type	Why
`const size = "large"`	`"large"`	a `const` can never hold anything else
`let size = "large"`	`string`	a later assignment of another string is legal
`const box = { size: "large" }`	`{ size: string }`	object properties behave like `let`

You can pin a literal down with an annotation (`let size: "large" = "large"`) or with `as const`, both of which are covered in chapter 8.

`null`, `undefined` and strictness

With `strictNullChecks` off, every type silently includes `null` and `undefined`, so nothing forces you to check. With it on, `string` means "a string, and nothing else", and `string | undefined` means "possibly missing". Optional properties and optional parameters are shorthand for the union form, which is why you must check them before use.

2.3 Examples

Example 1: the primitive types with annotations, run under type stripping.

TypeScript

```ts
const language: string = "TypeScript";
const major: number = 5;
const isTyped: boolean = true;
const notes: string[] = ["compiles to JS", "types are erased"];

console.log(language, major, isTyped, notes.length);
console.log(typeof language, typeof major, typeof isTyped);
```

output

```
TypeScript 5 true 2
string number boolean
```

Example 2: arrays and tuples, including a readonly one and a rest tuple.

TypeScript

```ts
const point: [number, number] = [4, 9];
const [x, y] = point;
console.log(x, y);

const pair: readonly [string, number] = ["id", 7];
console.log(pair[0], pair[1]);

const nested: [number, ...string[]] = [1, "a", "b"];
console.log(nested);
```

output

```
4 9
id 7
[ 1, 'a', 'b' ]
```

The printed form of the tuple is a plain array, because that is what it is once the annotation is erased.

Example 3: `unknown` and the narrowing it demands.

TypeScript

```ts
function describe(value: unknown): string {
  if (typeof value === "string") {
    return `string of length ${value.length}`;
  }
  if (typeof value === "number") {
    return `number, doubled ${value * 2}`;
  }
  return "something else";
}

console.log(describe("hi"));
console.log(describe(21));
console.log(describe(true));
```

output

```
string of length 2
number, doubled 42
something else
```

Inside each branch the compiler knows the narrower type, so `value.length` and `value * 2` are both legal. Outside the branches, neither would be.

Example 4: inference and widening.

TypeScript

```ts
const scores = [10, 20, 30];
console.log(scores.map((s) => s * 2));

const fixed = "north" as const;
let direction = "north";
console.log(fixed, direction);
direction = "south";
console.log(direction);
```

output

```
[ 20, 40, 60 ]
north north
south
```

`scores` is inferred as `number[]`, which is why `s * 2` needs no annotation, and `direction` is `string`, which is why reassigning it is allowed.

Example 5: optional members and the checks they force.

TypeScript

```ts
type User = { name: string; nickname?: string };

function greeting(user: User): string {
  return user.nickname ? `Hi ${user.nickname}` : `Hi ${user.name}`;
}

console.log(greeting({ name: "Ada" }));
console.log(greeting({ name: "Grace", nickname: "Gracey" }));
```

output

```
Hi Ada
Hi Gracey
```

`nickname?` means the property may be absent, so its type is `string | undefined` and using it without the check would be an error.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Reaching for `any` to make an error go away	It disables checking for that value and every value it flows into	Use `unknown` and narrow, or fix the annotation
Annotating every local variable	Doubles the noise and fights the inference you already have	Annotate parameters, returns and boundaries only
Forgetting that arrays need `T[]` not `T`	A single value and a list of values are different contracts	Let the compiler pick: `const list = [1, 2]` already knows it is `number[]`
Assuming `object` describes a shape	It only means "not a primitive" and accepts arrays and functions	Write the shape you actually expect
Expecting tuple positions to be optional	Accessing past the end of a tuple is an error, not `undefined`	Use an optional element or a rest element on purpose
Turning off `strictNullChecks` to avoid an error	You re-introduce every null bug the check exists to catch	Fix the code: check, default, or declare the union honestly
Mutating a `readonly` array in place	The compiler refuses, and at runtime the mutation is still visible to older code	Build a new array with spread, `concat` or `map`

2.5 Practice exercises

1. Write the type of a variable that holds a list of article titles, and the type of a function that takes a title and returns its length.
2. What is the difference between `any` and `unknown` when a value reaches a function you did not write, such as the result of `JSON.parse`?
3. A colleague writes `let mode = "strict"` and later assigns `mode = "loose"`. Why does the compiler accept it, and what would you change to forbid it?

4. Given `const pair: [string, number] = ["a", 1]`, what does `pair[2]` do at compile time and what does the same code produce at runtime?
5. Add types to this function so the body type-checks without a single cast: a function that takes an `unknown` value and returns its length as a number, or `0` when the value has no length.

Solutions

1. `const titles: string[] = []` and `function lengthOf(title: string): number { return title.length; }`.
2. `any` lets you use the value without any check, so a wrong assumption turns into a runtime crash. `unknown` forces you to prove the shape first - with `typeof`, `in`, `Array.isArray` or a custom guard - which is what you want for data you did not produce.
3. A literal assigned with `let` widens to `string`, so any other string is allowed. Write `let mode: "strict" | "loose" = "strict"` to keep the set of legal values small, as described in chapter 3.
4. The compiler reports an error: index 2 is outside a two-element tuple. At runtime the erased tuple is an ordinary array, so the expression evaluates to `undefined` unless the array was extended.
5. The typed version of the function for exercise 5:

TypeScript

```ts
function lengthOf(value: unknown): number {
  if (typeof value === "string" || Array.isArray(value)) {
    return value.length;
  }
  return 0;
}
```

No cast is needed, because `typeof` narrows `unknown` to `string` and `Array.isArray` narrows it to `any[]`, and both of those have `length`.

2.6 Mini test

1. Which annotation accepts any value but cannot be used until it is narrowed?
2. What type does the compiler infer for `const items = [1, "two"]`?
3. True or false: with `strictNullChecks` off, a `string` can also hold `null`.
4. What does `readonly` on a property change, and what does it not change?
5. Which two syntaxes write the same array type?
6. What is the inferred type of `let radius = 2`, and why is it not a literal type?
7. Which annotation should replace `object` when you want to describe a shape?

Answer key and explanations

1. `unknown`. It accepts every value but blocks member access until a guard narrows it.
2. `(string | number)[]`. The compiler takes the union of the element types it sees in the literal.

3. True. Without `strictNullChecks`, `null` and `undefined` are assignable to every type.
4. It prevents reassignment at compile time; it does not freeze or copy anything at runtime.
5. `T[]` and `Array<T>` are interchangeable.
6. `number`. A `let` variable can be assigned another number later, so the literal `2` widens to its base type.
7. A written object type, for example `{ radius: number }`.

Chapter 3 - Unions, literals and narrowing

3.1 Summary

- A union type lists the different kinds a value is allowed to hold, written with a vertical bar between the members.
- A union of literal types is the workhorse of typed code: `"asc" | "desc"` describes exactly two legal values and nothing else.
- Narrowing is the compiler following your checks and remembering which member of the union you are looking at.
- `typeof`, `instanceof`, `in`, `Array.isArray` and truthiness all narrow; a direct comparison against a literal narrows too.
- A discriminated union gives every member a shared property with a literal type, so a `switch` can pick the right branch without a cast.
- When every branch has been handled, the remaining type is `never`, which turns an exhaustiveness check into something the compiler verifies for you.
- Control-flow analysis is local: narrowing holds inside a block, and it can be forgotten inside a callback or after a function call.
- A user-defined type guard is a function whose return type is a predicate such as `value is Book`, so calling it narrows the argument.
- An assertion with `as` tells the compiler to stop checking; it removes errors rather than fixing them.
- With `strictNullChecks` on, optional values are unions with `undefined`, so narrowing is how you reach the real value.

3.2 Concepts

Writing unions

TypeScript

```ts
type Id = string | number;
type Direction = "north" | "south" | "east" | "west";
type Content = string | string[] | null;
```

A union is a set of possibilities. Operations that are legal on every member stay available; anything specific to one member is hidden until you narrow.

The guards

Check	What it narrows
`typeof x === "string"`	primitives: string, number, boolean, function, object, symbol, bigint
`x instanceof Class`	objects created by that class or its subclasses
`"prop" in x`	objects that have that property
`Array.isArray(x)`	arrays, including when the type is a union with a tuple
`if (x)`	removes `null`, `undefined`, `""`, `0`, `false` and `NaN`
`x === "literal"`	picks one member of a union of literals
`switch (x.kind)`	discriminated unions

output

```
value: string | number
       |
       |  typeof value === "string"
       +----------------> value: string      (only string left)
       |
       +----------------> value: number      (else branch)
```

Discriminated unions

Give each member a property holding a literal, and the compiler can split the union on that property alone:

TypeScript

```ts
type Circle = { kind: "circle"; radius: number };
type Square = { kind: "square"; side: number };
type Shape = Circle | Square;
```

Inside `case "circle"`, `shape.radius` exists. Inside `case "square"`, it does not - which is the point: a typo in a property name or a mixed-up member is caught immediately.

Exhaustiveness and `never`

After a `switch` handles every member of a finite union, the type left over is `never`. Assigning that to a `never` variable is a small trick that makes the compiler check you really covered everything:

TypeScript

```
function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.side ** 2;
    default: {
      const missing: never = shape; // compile error if a new member appears
      throw new Error("unhandled shape");
    }
  }
}
```

Add a third member to `Shape` later and this function stops compiling - the safest kind of refactoring error.

Where narrowing stops

Narrowing is control-flow analysis, so it follows the code path and nothing else. It does not survive:

- a function call that could have changed the value, when the value is a `let` or a mutable object property;
- a callback that runs later, such as an event handler or a `setTimeout`;
- an assertion that widened the type again (`as`).

Re-check inside a closure, or narrow once into a `const` you can trust.

Assertions

`as` is a promise you make to the compiler:

TypeScript

```
const input = document.querySelector("input") as HTMLInputElement;
```

The compiler drops its complaint; it does not verify the claim. A related operator, `as const`, is different: it does not remove checks, it makes a value more specific, as chapter 8 shows. Prefer narrowing and type guards; keep `as` for the cases where you know more than the checker can.

3.3 Examples

Example 1: a discriminated union with an exhaustiveness check.

TypeScript

```ts
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number };

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.side ** 2;
    default: {
      const never: never = shape;
      throw new Error(`unhandled shape ${JSON.stringify(never)}`);
    }
  }
}

console.log(area({ kind: "circle", radius: 1 }).toFixed(2));
console.log(area({ kind: "square", side: 3 }));
```

output

```
3.14
9
```

The `default` branch is unreachable at runtime today; it exists so that adding a third member to `Shape` becomes a compile error instead of a silent bug.

Example 2: `typeof` and `Array.isArray`.

TypeScript

```ts
function format(value: string | number | string[]): string {
  if (typeof value === "string") return value.trim();
  if (typeof value === "number") return value.toFixed(2);
  if (Array.isArray(value)) return value.join(", ");
  return "";
}

console.log(format("  hi  "));
console.log(format(3.14159));
console.log(format(["a", "b", "c"]));
```

output

```
hi
3.14
a, b, c
```

Each guard removes the members it cannot match, so the return statements need no casts.

Example 3: a union of results, tagged with a boolean literal.

TypeScript

```
type Success = { ok: true; data: string[] };
type Failure = { ok: false; error: string };
type Result = Success | Failure;

function report(result: Result): string {
  if (result.ok) {
    return `data: ${result.data.join("|")}`;
  }
  return `error: ${result.error}`;
}

console.log(report({ ok: true, data: ["one", "two"] }));
console.log(report({ ok: false, error: "timeout" }));
```

output

```
data: one|two
error: timeout
```

Example 4: `instanceof` and truthiness.

TypeScript

```
class HttpError extends Error {
  readonly status: number;
  constructor(status: number) {
    super(`HTTP ${status}`);
    this.status = status;
  }
}

function label(input: string | null | undefined): string {
  if (!input) return "empty";
  return input.toUpperCase();
}

console.log(label("ok"));
console.log(label(null));

try {
  throw new HttpError(404);
} catch (err) {
  if (err instanceof HttpError) {
    console.log(err.status, err.message);
  }
}
```

output

```
OK
empty
404 HTTP 404
```

`input` is `string | null | undefined`; `!input` removes all three falsy possibilities, leaving `string`. In the `catch` block, `err` is `unknown` in projects with `useUnknownInCatchVariables` on (part of `strict`), which is why the `instanceof` check is required.

Example 5: a user-defined type guard.

TypeScript

```ts
type Book = { kind: "book"; pages: number };
type Film = { kind: "film"; minutes: number };
type Media = Book | Film;

function isBook(media: Media): media is Book {
  return media.kind === "book";
}

function describe(media: Media): string {
  if (isBook(media)) {
    return `${media.pages} pages`;
  }
  return `${media.minutes} minutes`;
}

console.log(describe({ kind: "book", pages: 320 }));
console.log(describe({ kind: "film", minutes: 118 }));
```

output

```
320 pages
118 minutes
```

The return type `media is Book` is a predicate: it tells the compiler that a `true` result means the argument is a `Book`, so the call narrows just like a built-in guard. A guard that lies compiles fine and breaks at runtime, so keep its body honest.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using `as` to silence a union error	You told the compiler to stop checking, so the bug moves to production	Add a guard, or fix the type
Using `!` to skip a null check	It asserts instead of proving, and the crash returns as soon as the assumption fails	Check, default, or narrow with a guard
Repeating `typeof` checks that the compiler already did	Adds noise without adding safety	Trust control-flow analysis and read the hover type in the editor
Expecting narrowing to survive a closure	The compiler cannot prove when the callback runs, so it widens back	Narrow inside the callback, or copy into a `const` first
Using truthiness to test an optional number	`0` is falsy, so a valid zero takes the wrong branch	Compare explicitly: `value !== undefined` or `value !== null`
Forgetting `default` in a union `switch`	Adding a member silently does nothing instead of failing the build	Add an exhaustiveness branch with `never`

Mistake	Why it hurts	Best practice	
Writing `any` unions such as `string	any`	`any` swallows the whole union, so the type becomes `any`	Use `unknown` when the shape is genuinely open

3.5 Practice exercises

1. Write a union type for a value that is either a `number`, a `string`, or `undefined`, then write a function that returns `"missing"` for `undefined` and the value's length otherwise.
2. Why does the compiler refuse `shape.radius` when `shape: Shape` is a union of a circle and a square?
3. A `switch` over a union of three literal tags is missing one `case`. What compile-time trick makes that a build failure?
4. What is wrong with a type guard whose body always returns `true`?
5. After `const value = maybe as string`, what has the compiler actually done?

Solutions

1. `type Value = number | string | undefined;` and a function like the one below - the guard order matters, because `typeof undefined` is `"undefined"`.

TypeScript

```ts
function describe(value: number | string | undefined): string {
  if (value === undefined) return "missing";
  if (typeof value === "number") return `number ${value}`;
  return value.toUpperCase();
}
```

output

```
missing
number 42
Ada
```

2. Because the property exists only on one member of the union. A union exposes the members that are common to every possibility, so you must narrow to the circle first - a `switch` on `kind` is the usual way.
3. Assign the remaining value to a variable of type `never` in the `default` branch. While the union is exhausted the code compiles; as soon as a member is unhandled, the assignment fails and the build stops.
4. It lies. The predicate tells the compiler that a `true` result proves the argument is of that type, so every caller is now narrowing on a false assumption and no error is reported anywhere.
5. It accepted your claim without proof. The expression has type `string` from that line onwards, and any call that actually passes something else fails later, at runtime, with no compile-time warning.

3.6 Mini test

1. Which operator writes a union type, and what does `"a" | "b"` describe?
2. Which four built-in guards narrow a value in TypeScript?
3. What is a discriminated union?
4. Why is `never` useful in the `default` branch of a union `switch`?
5. True or false: narrowing inside an `if` block still applies to a callback defined in that block.
6. What does the return type `value is Book` do?
7. What is the difference between `as` and `as const`?

Answer key and explanations

1. The vertical bar. It describes a value that is either the string `"a"` or the string `"b"` and nothing else.
2. `typeof`, `instanceof`, `in` and `Array.isArray`, plus equality against a literal and plain truthiness.
3. A union whose members share a property holding a literal type, so a check on that property selects exactly one member.
4. `never` is what is left of a finite union once every member is handled, so assigning to it forces the compiler to confirm exhaustiveness.
5. False. A callback may run later, when the compiler can no longer prove anything about the outer variable, so the narrowing is dropped and you must re-check inside the callback.
6. It is a type predicate: a `true` result narrows the argument to `Book` at every call site, exactly like a built-in guard.
7. `as` asserts a type and suppresses checking; `as const` makes a value and its members as specific as possible, which adds precision instead of removing checks.

Chapter 4 - Functions and overloads

4.1 Summary

- Functions are values in JavaScript, so TypeScript needs a way to type the value and a way to type the call; it has both.
- A function type is a parameter list, an arrow and a return type: `(a: number, b: number) => number`.
- Parameters are typed positionally, and the return type is usually inferred when you leave it out.
- Optional parameters use `?`, default parameters take their type from the default, and rest parameters collect the remainder into an array.
- `void` describes a function whose result you should ignore; `never` describes one that never returns at all.
- Overloads let one function offer several call shapes: a set of bare signatures followed by one implementation signature that must accept all of them.

- Callbacks get their parameter types from the function you pass them to, which is why `array.map((x) => ...)` needs no annotation at all.
- Optional parameters must come last, and a `void` callback still means "the result is discarded", not "the callback must return nothing".
- Functions that return functions are how closures and factories are typed; the returned type is written after the arrow.
- When a signature is hard to read, name it with a type alias - it costs nothing at runtime and reads much better.

4.2 Concepts

One function, two descriptions

Notation	Describes
`function add(a: number, b: number): number`	the function declaration itself
`(a: number, b: number) => number`	the type of any function with that shape
`type Add = (a: number, b: number) => number`	a reusable name for that type

The type form is what you write for parameters, callbacks and object members.

Parameters

TypeScript

```ts
function build(slug: string, tags: string[] = [], locale: string = "en") {}
function collect(...ids: number[]) {}
function pick(key: string, fallback?: string) {}
```

Form	Rule
`name: type`	required
`name?: type`	optional; the type is really `type | undefined`
`name = value`	optional, with the type taken from the default
`...name: T[]`	rest; collects zero or more values into an array

Optional parameters must follow required ones. A parameter with a default is optional from the caller's point of view, even without a `?`.

Return types

Return types are inferred, and most teams annotate exported functions anyway - an explicit annotation stops an accidental change of return type from spreading through the codebase. `void` says "callers, ignore the result"; it does not force the function to return nothing, it only forbids using the value.

Overloads

Overloads describe several legal calls for one implementation:

TypeScript

```ts
function parse(input: string): number;
function parse(input: number): number;
function parse(input: string | number): number {
  return typeof input === "number" ? input : Number(input);
}
```

The first two lines are the callable signatures; the third is the implementation. Only the implementation body exists at runtime, so the caller sees the first two and the compiler checks that the implementation accepts everything the overloads promise. Rules worth remembering:

- the overload signatures have no bodies;
- the implementation signature must be compatible with each of them;
- the implementation signature is not callable from outside, so it is usually written with a wider type such as a union;
- order matters when two overloads could match: the compiler picks the first that fits.

Overloads are a tool for describing genuinely different call shapes. If the only difference is a single argument type, a union parameter is simpler.

Callbacks and contextual typing

When you pass a function literal where a function type is expected, the compiler types the parameters from that type - this is contextual typing:

TypeScript

```ts
const lengths = ["a", "bb"].map((s) => s.length); // s: string
```

The parameter `s` is inferred, so annotating it would be redundant. A callback typed as returning `void` may return anything; the caller simply ignores it, which is why `forEach` accepts an arrow that ignores its arguments.

4.3 Examples

Example 1: defaults, rest parameters and a return type.

TypeScript

```ts
function greet(name: string, greeting = "Hello"): string {
  return `${greeting}, ${name}!`;
}

function sum(...values: number[]): number {
  return values.reduce((a, b) => a + b, 0);
}

console.log(greet("Ada"));
console.log(greet("Grace", "Hi"));
console.log(sum(1, 2, 3));
```

output

```
Hello, Ada!
Hi, Grace!
6
```

`greeting` takes its type from the default `"Hello"`, so it is `string` and optional. `values` is `number[]`, so `sum()` with no arguments is legal too.

Example 2: a named function type used as a callback.

TypeScript

```ts
type Formatter = (value: number) => string;

function render(values: number[], format: Formatter): string[] {
  return values.map(format);
}

const money: Formatter = (value) => `$${value.toFixed(2)}`;
console.log(render([3.14159, 10], money));

const words = [1, 2, 3].map((n) => `item ${n}`);
console.log(words);
```

output

```
[ '$3.14', '$10.00' ]
[ 'item 1', 'item 2', 'item 3' ]
```

`value` in the `money` arrow is inferred from `Formatter`, and `n` in the `map` call is inferred from the array element type.

Example 3: overload signatures, erased before execution.

TypeScript

```ts
function makeId(prefix: string): string;
function makeId(value: number): string;
function makeId(input: string | number): string {
  return `${input}`.toUpperCase();
}

console.log(makeId("abc"));
console.log(makeId(42));
```

output

```
ABC
42
```

The two signature lines carry no runtime weight: Node's type stripping removes them and runs the implementation, which is why the second call prints `42` unchanged while the first is upper-cased.

Example 4: an optional result and optional chaining.

TypeScript

```ts
function find(items: string[], query: string): number | undefined {
  const index = items.indexOf(query);
  return index === -1 ? undefined : index;
}

const names = ["Ada", "Grace", "Linus"];
console.log(find(names, "Grace"));
console.log(find(names, "Alan"));
console.log(find(names, "Ada")?.toFixed(1));
```

output

```
1
undefined
0.0
```

The declared return type `number | undefined` forces every caller to deal with the missing case; `?.` is the shortest way to do it.

Example 5: a function that returns a function.

TypeScript

```ts
function multiplier(factor: number) {
  return (value: number): number => value * factor;
}

const triple = multiplier(3);
console.log(triple(5));
```

output

```
15
```

The returned type is `(value: number) => number`, so `triple` can only be called with a number. Writing that type out is optional here, because the compiler infers it.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Putting optional parameters before required ones	The compiler rejects it, and the call order becomes unreadable	Reorder, or give the earlier parameter a default
Annotating callback parameters that are already inferred	Duplicate noise that goes stale when the callback type changes	Delete the annotation and read the hover type
Using overloads for what a union parameter does fine	Overloads multiply the surfaces you must maintain	Write one signature with a union unless the calls are truly different
Making the implementation signature the only signature	Callers can then pass anything the union allows, defeating the point	Keep narrow overload signatures and a wide implementation

Mistake	Why it hurts	Best practice
Declaring a callback as returning `void` and then using the result	`void` means "the caller ignores it"; relying on the value is undefined behaviour	Use the real return type when you need the value
Forgetting `never` for a function that always throws	Callers think execution continues, so the compiler keeps types alive after the call	Return `never` and let the analysis narrow

4.5 Practice exercises

1. Write the type of a function called `compare` that takes two numbers and returns a number, then write a function `sort` that accepts it.
2. Add overloads to a function `format` so that it accepts a `Date` and returns an ISO string, and accepts a `number` and returns a string with two decimals.
3. What is the type of `x` in `[1, 2].map((x) => x.toFixed(1))`, and where does it come from?
4. Why does TypeScript allow a function typed `(cb: () => void) => void` to be called with a callback that returns a number?
5. Turn this into a typed signature with a rest parameter: `function concat(...parts) { return parts.join("-"); }`

Solutions

1. `type Compare = (a: number, b: number) => number;` and `function sort<T>(items: T[], compare: Compare): T[]` is not quite right for a generic list - the honest version is `function sortNumbers(items: number[], compare: Compare): number[]`.
2. The overloaded version, with one implementation body:

TypeScript

```typescript
function format(value: Date): string;
function format(value: number): string;
function format(value: Date | number): string {
  return value instanceof Date
    ? value.toISOString()
    : value.toFixed(2);
}
```

The two signatures are the callable faces; the third line is the implementation and is not visible to callers.

3. `number`. It is inferred from the element type of the array literal that `map` is called on - contextual typing flows from the method signature to the callback.
4. `void` describes what the caller does with the result, not what the callback is allowed to return. Allowing a value to be discarded is what makes `array.forEach((x) => list.push(x))` legal even though `push` returns a number. The typed version for exercise 5:

TypeScript

```ts
function concat(...parts: string[]): string {
  return parts.join("-");
}
```

4.6 Mini test

1. Which syntax writes the type of a function that takes a string and returns a boolean?
2. What is the difference between `name?: string` and `name = "x"`?
3. How many function bodies may an overload set have?
4. What does a `void` return type on a callback actually promise?
5. True or false: the implementation signature of an overloaded function is callable from outside the module.
6. Where does the type of a `.map` callback parameter come from?
7. What does `never` as a return type tell the compiler?

Answer key and explanations

1. `(value: string) => boolean`, usually given a name with a type alias.
2. `?:` makes the parameter optional and adds `| undefined` to its type; a default also makes it optional but the parameter always has a value inside the body.
3. Exactly one. The other signatures are declarations without bodies.
4. That the caller will ignore the result; the callback itself may return anything.
5. False. Only the overload signatures are visible to callers, which is why the implementation is written with a wider type such as a union.
6. From the array's element type, through the contextual type of `map`.
7. That the function never returns normally - it throws or loops forever - so the compiler can treat the code after the call as unreachable.

Chapter 5 - Interfaces, type aliases and object shapes

5.1 Summary

- Object types describe the shape of a value: which properties it has, what each holds, and which are optional or read-only.
- An `interface` and a type alias for an object type are almost interchangeable; the differences show up in merging, in errors and in `extends` versus `&`.
- Optional properties use `?` and add `undefined`; `readonly` properties may be assigned at creation and never again.
- Index signatures describe a dictionary where every key holds the same type, while a mapped type transforms every property of an existing type.

- Declaration merging means two interfaces with the same name in the same scope combine into one; type aliases cannot merge.
- `extends` copies a base shape into a new interface, and an intersection (`&`) combines two shapes into one.
- Interfaces can describe callable and constructable shapes, so functions and classes can be typed as objects too.
- Excess property checking catches a misspelled property in an object literal assigned to a known type - a small rule that prevents a surprising class of typos.
- `interface` is the friendlier choice for objects that other code may extend or augment, including declaration merging; aliases win for unions, tuples and computed shapes.
- Whatever you choose, describe the shape you need rather than adding types the code does not use.

5.2 Concepts

Writing a shape

TypeScript

```typescript
interface Book {
  readonly isbn: string;
  title: string;
  author: string;
  pages?: number;
}
```

Member	Meaning
`title: string`	required, a string
`pages?: number`	may be missing; its type is `number | undefined`
`readonly isbn: string`	assignable once, at creation
`summary(): string`	a method that returns a string
`[key: string]: unknown`	any additional property is allowed, of that type

`readonly` is a compile-time promise. The object is not frozen, so a plain JavaScript consumer can still write to it.

Interface or type alias?

Question	`interface`	`type`
Describes an object shape	yes	yes
Describes a union or tuple	no	yes
Can be merged with another declaration of the same name	yes	no
Extends by copying members	`extends`	intersection `&`

Question	interface	type
Works with `implements` on a class	yes	yes
Appears in error messages as	the interface name	the written-out type

The practical guidance: use `interface` for the objects your code owns and other code may extend, and `type` for everything computed - unions, tuples, mapped and conditional types, and function types.

Extending and intersecting

`extends` lists one or more bases:

TypeScript

```ts
interface Entry extends Base, Timestamps {
  label: string;
}
```

An intersection combines two complete types:

TypeScript

```ts
type Tagged = { id: number } & { label: string };
```

An intersection of two object types has all the members of both. Where they disagree, the result is often an impossible type, which is why unions of shapes are usually safer than intersecting unrelated ones.

Index signatures

TypeScript

```ts
type Dict = { [key: string]: number };
```

Every access through a string key returns `number`, and assigning a string key is always allowed. With `noUncheckedIndexedAccess` enabled the compiler adds `| undefined`, which is closer to reality. An index signature says "I do not know the keys in advance"; when you do know them, list them.

Excess property checking

Assigning a fresh object literal to a known type checks the property names, so a typo is an error:

TypeScript

```ts
const book: Book = { titel: "..." }; // error: 'titel' does not exist in type 'Book'
```

This check applies to literals only. Once a value is stored in a variable, the compiler trusts its declared type - which is why widening a variable too early hides typos.

5.3 Examples

Example 1: an interface with optional and read-only members.

TypeScript

```typescript
interface Book {
  readonly isbn: string;
  title: string;
  author: string;
  pages?: number;
}

const book: Book = {
  isbn: "978-0-13",
  title: "Structure and Interpretation",
  author: "Abelson",
  pages: 683,
};

console.log(`${book.title} by ${book.author}`);
console.log(book.pages ?? "unknown");
console.log(book.isbn);
```

output

```
Structure and Interpretation by Abelson
683
978-0-13
```

`pages` is optional, so the `??` is required before printing it; `isbn` cannot be reassigned anywhere in the code.

Example 2: a type alias with a union and an intersection.

TypeScript

```typescript
type Id = string | number;
type Named = { name: string };

function label(item: Named & { id: Id }): string {
  return `${item.name}#${item.id}`;
}

console.log(label({ name: "ada", id: 42 }));
console.log(label({ name: "grace", id: "g7" }));
```

output

```
ada#42
grace#g7
```

Example 3: an index signature as a dictionary.

TypeScript

```ts
type Dict = { [key: string]: number };

function tally(votes: string[]): Dict {
  const result: Dict = {};
  for (const vote of votes) {
    result[vote] = (result[vote] ?? 0) + 1;
  }
  return result;
}

console.log(tally(["a", "b", "a", "a", "b"]));
```

output

```
{ a: 3, b: 2 }
```

The `?? 0` matters: with a strict index signature the first lookup for a key is typed as possibly missing, so the accumulation starts from a real `0`.

Example 4: `extends` and declaration merging in one file.

TypeScript

```ts
interface Base {
  id: number;
}
interface Timestamps {
  createdAt: string;
}
interface Entry extends Base, Timestamps {
  label: string;
}

interface Config {
  retries: number;
}
interface Config {
  timeout: number;
}

const entry: Entry = { id: 1, createdAt: "2026-10-10", label: "first" };
console.log(entry.label, entry.id, entry.createdAt);

const config: Config = { retries: 3, timeout: 5000 };
console.log(config.retries, config.timeout);
```

output

```
first 1 2026-10-10
3 5000
```

`Entry` inherits `id` and `createdAt`, and the two `Config` declarations merged into one shape, so an object must supply both properties.

Example 5: a function type written as an interface member and as an alias.

TypeScript

```ts
interface Comparator<T> {
  (a: T, b: T): number;
  label: string;
}

const byLength: Comparator<string> = (a, b) => a.length - b.length;
byLength.label = "length";

console.log(["bb", "a", "ccc"].sort(byLength));
console.log(byLength.label);
```

output

```
[ 'a', 'bb', 'ccc' ]
length
```

An interface with a bare call signature describes a callable object that also has properties - the shape of a function with metadata attached.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Re-declaring a type alias with the same name	Duplicate identifier error; only interfaces merge	Use an interface when merging is the intent
Using `interface` for a union	Not possible at all, so people fall back to `any`	Use a `type` alias for unions and tuples
Adding an index signature to a fixed shape	It silently allows unknown keys and hides typos	List the properties; add an index signature only for real dictionaries
Mutating a `readonly` property through a cast	The cast removes the only guard you had	Copy the object with spread and write a new value
Writing `Partial`-like optional properties on a type used for input	Callers can omit required data by accident	Separate input and output shapes
Storing an object literal in an untyped variable first	Excess property checking never runs, so a misspelling survives	Annotate at the point of construction

5.5 Practice exercises

1. Describe a `Comment` with a required `text`, an optional `rating` and a read-only `author`, then write a function that returns a string that includes the rating only when present.
2. Rewrite the following as a discriminated union of interfaces: a value that is either `{ kind: "text"; value: string }` or `{ kind: "size"; px: number }`.
3. What is the difference between `extends` and `&`, and when does each read better?
4. Which declaration allows a second interface with the same name to add properties, and why is that useful for augmenting a library?

5. Given `type Env = { [key: string]: string }`, why is `env.PORT` a risky access without `noUncheckedIndexedAccess`?

Solutions

1. The interface, with the optional member last:

TypeScript

```ts
interface Comment {
  readonly author: string;
  text: string;
  rating?: number;
}
```

A formatter that checks the optional member before using it:

TypeScript

```ts
function show(comment: Comment): string {
  const base = `${comment.author}: ${comment.text}`;
  return comment.rating === undefined ? base : `${base} (${comment.rating})`;
}
```

The check is what makes `comment.rating` usable; without it the compiler refuses to print a value that may not exist.

The union of two interfaces, for exercise 2:

TypeScript

```ts
type TextNode = { kind: "text"; value: string };
type SizeNode = { kind: "size"; px: number };
type Node = TextNode | SizeNode;
```

The `kind` property is the discriminant, so a `switch` on it selects one member without a cast.

3. `extends` copies members into a new interface and keeps the name readable in error messages; `&` builds a new type from two complete ones and can express combinations that no interface names. Use `extends` for an "is a" relationship, `&` to glue two shapes together at a call site.
4. `interface`. Merging is how a project adds properties to a shape declared elsewhere - for example adding a field to a library's request object - without editing the original declaration. A type alias cannot merge, so a second declaration is a duplicate identifier error.
5. Because the index signature promises a `string` for every key, so a missing `PORT` looks like a present one. With `noUncheckedIndexedAccess` the compiler types the result as `string | undefined` and forces a check.

5.6 Mini test

1. Which member declaration makes a property possibly missing?
2. What does `readonly` change at compile time, and what does it not change?
3. Can two type aliases with the same name merge?

4. What is an index signature for?
5. Which check fires when an object literal has a property that is not in the target type?
6. When is a `type` alias required rather than an `interface`?
7. What does `interface Entry extends Base, Timestamps` mean for `Entry`'s members?

Answer key and explanations

1. `name?: type`.
2. It forbids reassignment after creation; it does not freeze the object or change runtime behaviour.
3. No. Only interfaces merge; a repeated alias is a duplicate identifier.
4. Describing a dictionary whose keys are not known in advance but whose values share one type.
5. Excess property checking, which applies only to fresh object literals.
6. When the type is a union, a tuple, a function type or anything computed from other types.
7. `Entry` has every member of `Base` and of `Timestamps` plus its own, so an object of that type must supply all of them.